Rajalakshmi Engineering College

Name: Keerthana G
Email: 240701254@rajalakshmi.edu.in
Roll no: 240701254
Phone: 9342478755
Branch: REC
Department: CSE - Section 3
Batch: 2028
Degree: B.E - CSE

Scan to verify results

2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 9_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 11

Section 1 : MCQ

1. What is Collection in Java?

Answer

A group of interfaces

Status : Wrong ***Marks : 0/1***

2. What will be the output of the following code?

```
import java.util.*;
class Main {
   public static void main(String[] args) {
      ArrayList<Integer> list = new ArrayList<>();
       list.add(1);
       list.add(2);
```

240701254

```
        list.add(3);
        list.add(4);
        list.add(5);
        System.out.println(list.get(3));
    }
}
```

Answer

4

Status : Correct ***Marks : 1/1***

3. What will be the output of the following code?

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> list = new ArrayList<>();
        list.add("Apple");
        list.add("Banana");
        list.remove("Apple");
        System.out.println(list);

    }
}
```

Answer

[Banana]

Status : Correct ***Marks : 1/1***

4. Which method is used to add an element to the top of the stack?

Answer

push()

Status : Correct ***Marks : 1/1***

240701254

5. What will be the output of the following code?

```
import java.util.*;
class Main {
   public static void main(String[] args) {
      ArrayList<String> list = new ArrayList<>();
      list.add("apple");
      list.add("banana");
      list.add("cherry");
      list.add("banana");
      System.out.println(list.lastIndexOf("banana"));
   }
}
```

Answer

4

Status : Wrong ***Marks : 0/1***

6. What will be the output of the following code?

```
import java.util.ArrayList;

public class Main {
   public static void main(String[] args) {
      ArrayList<Integer> list = new ArrayList<>();
      list.add(10);
      list.add(20);
      list.add(30);
      System.out.println("Size of the list: " + list.size());
   }
}
```

Answer

Size of the list: 3

Status : Correct ***Marks : 1/1***

7. What will be the output of the following code?

```
import java.util.*;
class Main {
    public static void main(String[] args) {
        ArrayList<String> list = new ArrayList<>();
        list.add("Java");
        list.add("Python");
        list.add("Java");
        list.add("C++");
        System.out.println(list.indexOf("Java"));
    }
}
```

Answer

1

Status : Wrong ***Marks : 0/1***

8. What will be the output of the following code?

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Stack<Integer> stack = new Stack<>();
        for (int i = 1; i <= 3; i++)
            stack.push(i * 2);
        stack.pop();
        stack.push(10);
        System.out.println(stack.peek());
    }
}
```

Answer

10

Status : Correct ***Marks : 1/1***

9. What will be the output of the following code?

240701254

```
import java.util.*;
class Main {
   public static void main(String[] args) {
      ArrayList<Integer> list = new ArrayList<>();
      list.add(1);
      list.add(2);
      list.add(3);
      list.add(4);
      list.set(2, 10);
      System.out.println(list);
   }
}
```

Answer

[1, 2, 10, 4]

Status : Correct ***Marks : 1/1***

10. What will be the output of the following code?

```
import java.util.*;
public class Main {
   public static void main(String[] args) {
      Stack<Integer> s = new Stack<>();
      s.push(10);
      s.push(20);
      s.push(30);
      System.out.println(s.peek());
   }
}
```

Answer

30

Status : Correct ***Marks : 1/1***

11. What will be the output of the following code?

```
import java.util.*;
```

240701254

```
class Main {
    public static void main(String[] args) {
        ArrayList<Integer> list = new ArrayList<>();
        list.add(10);
        list.add(20);
        list.add(30);
        list.remove(1);
        System.out.println(list);
    }
}
```

Answer

[10, 20, 30]

Status : Wrong ***Marks : 0/1***

12. Which of the following methods removes and returns the last element from a LinkedList?

Answer

removeLast()

Status : Correct ***Marks : 1/1***

13. What is the correct way to create an ArrayList in Java?

Answer

ArrayList<String> list = new ArrayList<>();

Status : Correct ***Marks : 1/1***

14. What does the addFirst() method of LinkedList do?

Answer

Adds an element to the beginning of the list

Status : Correct ***Marks : 1/1***

240701254

15. How can you access the first element of an ArrayList named as list?

Answer

list.get(0);

Status : Correct ***Marks : 1/1***

240701254

Rajalakshmi Engineering College

Name: Keerthana G
Email: 240701254@rajalakshmi.edu.in
Roll no: 240701254
Phone: 9342478755
Branch: REC
Department: CSE - Section 3
Batch: 2028
Degree: B.E - CSE

Scan to verify results

2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 9_Q1

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Bobby is tasked with processing a sequence of numbers from a monitoring system. He needs to extract a strictly increasing subsequence using an ArrayList. The program should dynamically add numbers to the ArrayList only if they are greater than the last number currently stored in the list. Bobby aims to efficiently utilize the dynamic resizing and indexing features of the ArrayList to solve this problem.

Help Bobby implement this solution.

Input Format

The first line of input consists of an integer N, representing the number of elements.

240701254

The second line consists of N space-separated integers, representing the elements.

Output Format

The output prints the list of integers in increasing sequence, ignoring out-of-order elements.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 7
3 5 9 1 11 7 13
Output: [3, 5, 9, 11, 13]

Answer

```
// You are using Java
import java.util.ArrayList;
import java.util.Scanner;

public class Main

{

  public static void main(String[] args)

{


      Scanner sc = new Scanner(System.in);
      int N = sc.nextInt();
      int[] arr = new int[N];
      for (int i = 0; i < N; i++)

{


          arr[i] = sc.nextInt();
```

240701254

```
}
        ArrayList<Integer> increasingList = new ArrayList<>();
        for (int i = 0; i < N; i++)
{
            if (increasingList.isEmpty() || arr[i] >
increasingList.get(increasingList.size() - 1))
{
                increasingList.add(arr[i]);
}
}
        System.out.println(increasingList);
}
}
```

Status : Correct ***Marks : 10/10***

Rajalakshmi Engineering College

Name: Keerthana G
Email: 240701254@rajalakshmi.edu.in
Roll no: 240701254
Phone: 9342478755
Branch: REC
Department: CSE - Section 3
Batch: 2028
Degree: B.E - CSE

Scan to verify results

2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 9_Q2

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Vikram loves listening to music and wants to create a simple playlist manager using Java Collections. The playlist supports the following operations:

"ADD <song>" Adds the song to the end of the playlist."REMOVE <song>" Removes the first occurrence of the song from the playlist. If the song is not found, do nothing."SHOW" Displays all songs in the playlist in order. If the playlist is empty, print "EMPTY"."NEXT" Moves to the next song in the playlist and prints its name. If the playlist is empty, print "EMPTY".

The playlist maintains a "current song" position that starts at the first song when it's added. The NEXT command moves to the next song and prints it, wrapping around to the first song after reaching the last song. When removing songs, the current position adjusts accordingly to maintain

proper navigation.

Help Vikram implement this playlist manager.

Input Format

The first line of the input consists of an integer n, the number of operations.

The next n lines, each containing a command:

- "ADD <song>"
- "REMOVE <song>"
- "SHOW"
- "NEXT"

Output Format

For each "SHOW" command, print the songs in order, separated by spaces.

For each "NEXT" command, print the next song in the playlist.

If no song exists, print "EMPTY".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7
ADD song1
ADD song2
SHOW
NEXT
REMOVE song2
SHOW
NEXT

Output: song1 song2
song2
song1
song1

Answer

240701254

```
// You are using Java
import java.util.LinkedList;
import java.util.Scanner;

public class Main

{


   public static void main(String[] args)

{


      Scanner sc = new Scanner(System.in);
      int n = sc.nextInt();
      sc.nextLine();

      LinkedList<String> playlist = new LinkedList<>();
      int currentIndex = -1;

      for (int i = 0; i < n; i++)

{


         String command = sc.nextLine();

         if (command.startsWith("ADD "))

{


            String song = command.substring(4);
            playlist.add(song);
            if (currentIndex == -1) currentIndex = 0;


} else if (command.startsWith("REMOVE "))

{
```

```
            String song = command.substring(7).trim();
            int indexToRemove = playlist.indexOf(song);
            if (indexToRemove != -1)

{

                playlist.remove(indexToRemove);
                if (playlist.isEmpty())

{

                    currentIndex = -1;

} else if (indexToRemove <= currentIndex)

{

                    currentIndex--;
                    if (currentIndex < 0) currentIndex = 0;

}

}

} else if (command.equals("SHOW"))

{

            if (playlist.isEmpty())

{
```

```
            System.out.println("EMPTY");

    } else

    {

            for (int j = 0; j < playlist.size(); j++)

    {

                System.out.print(playlist.get(j));
                if (j != playlist.size() - 1) System.out.print(" ");

    }
            System.out.println();

    }

    } else if (command.equals("NEXT"))

    {

          if (playlist.isEmpty())

    {

            System.out.println("EMPTY");

    } else

    {
```

240701254

```
            currentIndex = (currentIndex + 1) % playlist.size();
            System.out.println(playlist.get(currentIndex));

    }

    }

    }

    }
}
```

Status : Correct ***Marks : 10/10***

Rajalakshmi Engineering College

Name: Keerthana G
Email: 240701254@rajalakshmi.edu.in
Roll no: 240701254
Phone: 9342478755
Branch: REC
Department: CSE - Section 3
Batch: 2028
Degree: B.E - CSE

Scan to verify results

2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 9_Q3

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Assist Pranitha in developing a program that takes an integer N as input, representing the number of names to be read. Then read N names and store them in an ArrayList. Finally, input a search string and output the frequency of that string in the list of names.

Note: Some parts of the code are provided as snippets, and you need to complete the remaining sections by writing the necessary code.

Input Format

The first line of input consists of an integer N, representing the number of names to be read.

The following N lines consist of N names, as a string.

240701254

The last line consists of a string, representing the name to be searched.

Output Format

The output prints a single integer, representing the frequency of the specified name in the given list.

If the specified name is not found, print 0.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
Alice
Bob
Ankit
Alice
Pranitha
Alice

Output: 2

Answer

```
// You are using Java
import java.util.ArrayList;
import java.util.Scanner;

public class Main

{


   public static void main(String[] args)

{


      Scanner sc = new Scanner(System.in);
      int N = sc.nextInt();
      sc.nextLine();
```

240701254

```
        ArrayList<String> names = new ArrayList<>();
        for (int i = 0; i < N; i++)
{
            names.add(sc.nextLine());
}
        String searchName = sc.nextLine();
        int count = 0;
        for (String name : names)
{
            if (name.equals(searchName))
{
                count++;
}
}
        System.out.println(count);
}
}
```

Status : Correct ***Marks : 10/10***

Rajalakshmi Engineering College

Name: Keerthana G
Email: 240701254@rajalakshmi.edu.in
Roll no: 240701254
Phone: 9342478755
Branch: REC
Department: CSE - Section 3
Batch: 2028
Degree: B.E - CSE

Scan to verify results

2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 9_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 38.5

Section 1 : Coding

1. Problem Statement

A teacher is filtering a list of words provided by students. Some words contain too many vowels, making them difficult for a spelling competition. The teacher decides to remove all words that contain more than two vowels.

Help the teacher to implement it using ArrayList.

Input Format

The first line contains an integer N, representing the number of words in the list.

The next N lines contain a string representing the words (one per line).

Output Format

240701254

The output consists of words that contain two or less than two vowels, printed in the same order they appeared in the input. Each word is printed on a new line.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 1
sri

Output: sri

Answer

```
import java.util.ArrayList;
import java.util.Scanner;

class VowelFilter {
    public static void filterWords(int n, Scanner sc) {
        ArrayList<String> words = new ArrayList<>();

        for (int i = 0; i < n; i++) {
            words.add(sc.next());
        }

        for (String word : words) {
            int vowelCount = 0;
            for (char c : word.toCharArray()) {
                if (c=='a' || c=='e' || c=='i' || c=='o' || c=='u') {
                    vowelCount++;
                }
            }

            if (vowelCount <= 2) {
                System.out.println(word);
            }
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```
        int n = sc.nextInt();
        sc.nextLine();
        VowelFilter.filterWords(n, sc);
        sc.close();
    }
}
```

Status : Partially correct ***Marks : 8.5/10***

2. Problem Statement

Sarah, a warehouse manager, is managing a list of product names in her store's inventory system. She needs to perform basic operations like adding (inserting) new products, removing products that are sold out or discontinued, displaying all the products in stock, and searching for a specific product in the inventory list.

Sarah's goal is to manage the inventory using a list of product names (strings). The system allows her to perform the following operations using ArrayList:

Insert a Product: Sarah adds a new product to the inventory.Delete a Product: Sarah removes a product from the inventory when it's sold or discontinued.Display the Inventory: Sarah checks all the products currently available in the inventory.Search for a Product: Sarah searches for a specific product in the inventory to check if it's available.

Input Format

The input consists of multiple space-separated values representing different operations on a product list. Each operation follows a specific format:

1 <product_name> - Adds <product_name> to the product list.

2 <product_name> - Removes <product_name> from the product list if it exists.

3 - Print all products currently on the list.

4 <product_name> - Checks if <product_name> exists in the list.

Output Format

The output displays,

For (choice 1) prints, " <item> has been added to the list."

For (choice 2) prints, " <item> has been removed from the list."

For (choice 3) prints, "Items in the list:" followed by each item in the list on a new line, or "The list is empty." if the list is empty.

For (choice 4) prints," <item> is found in the list." or " <item> not found in the list."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1 apple 1 banana 2 apple 3 4 apple

Output: apple has been added to the list.
banana has been added to the list.
apple has been removed from the list.
Items in the list:
banana
apple not found in the list.

Answer

```
import java.util.ArrayList;
import java.util.Scanner;

class StringListOperations {

    public static void insertItem(ArrayList<String> list, String item) {
        list.add(item);
        System.out.print(item + " has been added to the list. ");
    }

    public static void deleteItem(ArrayList<String> list, String item) {
        if (list.contains(item)) {
            list.remove(item);
            System.out.print(item + " has been removed from the list. ");
        } else {
            System.out.print(item + " not found in the list. ");
```

```
        }
    }

    public static void displayList(ArrayList<String> list) {
        if (list.isEmpty()) {
            System.out.print("The list is empty. ");
        } else {
            System.out.print("Items in the list: ");
            for (String s : list) {
                System.out.print(s + " ");
            }
        }
    }

    public static void searchItem(ArrayList<String> list, String item) {
        if (list.contains(item)) {
            System.out.print(item + " is found in the list. ");
        } else {
            System.out.print(item + " not found in the list. ");
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        ArrayList<String> list = new ArrayList<>();

        String input = sc.nextLine();
        String[] commands = input.split(" ");
        int i = 0;
        while (i < commands.length) {
            int choice = Integer.parseInt(commands[i]);
            switch (choice) {
                case 1:
                    if (i + 1 < commands.length) {
                        StringListOperations.insertItem(list, commands[i + 1]);
                        i += 2;
                    } else {
                        System.out.println("No string provided for insertion.");
                        i++;
                    }
                    break;
```

```
            case 2:
                if (i + 1 < commands.length) {
                    StringListOperations.deleteItem(list, commands[i + 1]);
                    i += 2;
                } else {
                    System.out.println("No string provided for deletion.");
                    i++;
                }
                break;
            case 3:
                StringListOperations.displayList(list);
                i += 1;
                break;
            case 4:
                if (i + 1 < commands.length) {
                    StringListOperations.searchItem(list, commands[i + 1]);
                    i += 2;
                } else {
                    System.out.println("No string provided for searching.");
                    i++;
                }
                break;
        }
    }
  }
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Rahul is working on a list manipulation problem where he needs to reverse a specific subarray using a stack. Given an array and two indices l and r, he wants to reverse only the portion of the array from index l to r (both inclusive) while keeping the rest of the array unchanged.

Since Rahul wants to solve this problem efficiently, he decides to use a stack to reverse the subarray in O(r - l) time.

Your task is to help Rahul by implementing this functionality.

Input Format

The first line contains an integer n, the size of the array.

The second line contains n space-separated integers arr[i].

The third line contains two integers l and r, denoting the start and end indices of the subarray to reverse.

Note: The array follows 0-based indexing.

Output Format

The output prints the modified array after reversing the subarray between indices l and r.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6
1 2 3 4 5 6
1 4
Output: 1 5 4 3 2 6

Answer

```
import java.util.*;

public class Main {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        for(int i = 0; i < n; i++){
            arr[i] = sc.nextInt();
        }
```

240701254

```
        int l = sc.nextInt();
        int r = sc.nextInt();

        // Using stack to reverse subarray
        java.util.Stack<Integer> stack = new java.util.Stack<>();

        for(int i = l; i <= r; i++){
            stack.push(arr[i]);
        }

        for(int i = l; i <= r; i++){
            arr[i] = stack.pop();
        }

        for(int i = 0; i < n; i++){
            System.out.print(arr[i]);
            if(i < n - 1) System.out.print(" ");
        }
    }

}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Sanjay is working on a program to merge two sorted linked lists into a single sorted list using Java's LinkedList class from the Collections framework. Given two sorted linked lists, he wants to merge them while maintaining the sorted order.

Write a Java program that:

Reads two sorted linked lists.Merges them into a single sorted linked list.Prints the merged list in ascending order.

Input Format

The first line contains an integer m (the size of the first linked list).

The second line contains m space-separated integers (sorted).

The third line contains an integer n (the size of the second linked list).

The fourth line contains n space-separated integers (sorted).

Output Format

The output prints the merged linked list as space-separated integers.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 2
5 10
3
1 3 8
Output: 1 3 5 8 10

Answer

```
import java.util.*;
class MergeSortedLinkedLists {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        int m = scanner.nextInt();
        LinkedList<Integer> list1 = new LinkedList<>();
        for (int i = 0; i < m; i++) {
            list1.add(scanner.nextInt());
        }

        int n = scanner.nextInt();
        LinkedList<Integer> list2 = new LinkedList<>();
        for (int i = 0; i < n; i++) {
            list2.add(scanner.nextInt());
        }

        LinkedList<Integer> merged = new LinkedList<>();
        int i = 0, j = 0;
```

```
        while (i < list1.size() && j < list2.size()) {
            if (list1.get(i) <= list2.get(j)) {
                merged.add(list1.get(i));
                i++;
            } else {
                merged.add(list2.get(j));
                j++;
            }
        }

        while (i < list1.size()) {
            merged.add(list1.get(i));
            i++;
        }

        while (j < list2.size()) {
            merged.add(list2.get(j));
            j++;
        }

        for (int num : merged) {
            System.out.print(num + " ");
        }

        scanner.close();
    }
}
```

Status : Correct ***Marks : 10/10***

Rajalakshmi Engineering College

Name: Keerthana G
Email: 240701254@rajalakshmi.edu.in
Roll no: 240701254
Phone: 9342478755
Branch: REC
Department: CSE - Section 3
Batch: 2028
Degree: B.E - CSE

Scan to verify results

2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 9_PAH

Attempt : 1
Total Mark : 30
Marks Obtained : 30

Section 1 : Coding

1. Problem Statement

Rekha is a teacher who wants to calculate the average of marks scored by her students in a test. She needs to store all the marks dynamically because the number of students may vary each time. Using an ArrayList allows her to easily add any number of marks without worrying about the initial size.

Help her implement the task.

Input Format

The first line of input is an integer n, representing the number of students..

The second line of input consists of n double values, representing the marks of each student, separated by a space.

240701254

Output Format

The output prints: "Average of the list: " followed by the average value formatted to two decimal places.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
1.0 2.0 3.0 4.0 5.0

Output: Average of the list: 3.00

Answer

```
// You are using Java
import java.util.ArrayList;
import java.util.Scanner;

public class Main

{


   public static void main(String[] args)

{


      Scanner sc = new Scanner(System.in);
      int n = sc.nextInt();
      ArrayList<Double> marksList = new ArrayList<>();

      for (int i = 0; i < n; i++)

{


         double mark = sc.nextDouble();
         marksList.add(mark);
```

240701254

```
}
        double sum = 0;
        for (double mark : marksList)
{

            sum += mark;

}
        double average = sum / n;
        System.out.printf("Average of the list: %.2f%n", average);

}

}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Arun is building a task manager to keep track of tasks using a LinkedList. The task manager supports the following operations:

"ADD <task>" Adds the given task to the end of the list."REMOVE" Removes the first task from the list."SHOW" Displays all tasks in the list in order. If the list is empty, print "EMPTY".

Help Arun implement this functionality using a LinkedList.

Input Format

The first line of the input consists of an integer n, the number of operations.

The next n lines, each containing a command:

240701254

- "ADD <task>"
- "REMOVE"
- "SHOW"

Output Format

For each "SHOW" command, the output prints the tasks in order, separated by spaces.

If no tasks exist, print "EMPTY".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
ADD homework
ADD project
SHOW
REMOVE
SHOW

Output: homework project
project

Answer

```
// You are using Java
import java.util.LinkedList;
import java.util.Scanner;

public class Main

{


    public static void main(String[] args)

{


        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
```

```
        sc.nextLine(); // consume newline
        LinkedList<String> tasks = new LinkedList<>();
        for (int i = 0; i < n; i++)
    {

            String command = sc.nextLine();
            if (command.startsWith("ADD "))
    {

                tasks.add(command.substring(4));

    } else if (command.equals("REMOVE"))
    {

                if (!tasks.isEmpty())
    {

                    tasks.removeFirst();

    }

    } else if (command.equals("SHOW"))
    {

                if (tasks.isEmpty())
```

```
{
            System.out.println("EMPTY");
} else
{
            System.out.println(String.join(" ", tasks));
}
}
}
}
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Aditi is analyzing stock market trends and wants to find the Next Greater Element (NGE) for each stock price in a list. The Next Greater Element for an element x in an array is the first element to the right that is greater than x. If no greater element exists, return -1 for that position.

Your task is to help Aditi by efficiently computing the Next Greater Element for each element in the given array using a Stack.

Example:

Input:

6

4 5 2 10 8 6

Output:

5 10 10 -1 -1 -1

Explanation:

For each element:

4 5 (next greater element)5 102 1010 -1 (No greater element)8 -16 -1

Input Format

The first line contains an integer n, representing the number of elements.

The second line contains n space-separated integers arr[i], where arr[i] is the stock price on the i-th day.

Output Format

The output prints n space-separated integers representing the Next Greater Element for each element in the array.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6
4 5 2 10 8 6

Output: 5 10 10 -1 -1 -1

Answer

```
// You are using Java
import java.util.Scanner;
import java.util.Stack;

public class Main

{
```

240701254

```
public static void main(String[] args)
{
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();
    int[] arr = new int[n];
    int[] nge = new int[n];

    for (int i = 0; i < n; i++)
{
        arr[i] = sc.nextInt();

}
    Stack<Integer> stack = new Stack<>();

    for (int i = n - 1; i >= 0; i--)
{
        while (!stack.isEmpty() && stack.peek() <= arr[i])
{
            stack.pop();

}
        nge[i] = stack.isEmpty() ? -1 : stack.peek();
        stack.push(arr[i]);
```

```
}
        for (int i = 0; i < n; i++)
{

            System.out.print(nge[i] + (i == n - 1 ? "\n" : " "));

}

}
}
```

Status : Correct ***Marks : 10/10***